

From FCFS to EEVDF: An Empirical Reconstruction of the Linux Scheduler Evolution

Course Project Team
CS 3250 – Operating Systems
Spring 2026

May 17, 2026

Abstract

Linux replaced the Completely Fair Scheduler (CFS) with the Earliest Eligible Virtual Deadline First (EEVDF) scheduler in v6.6 [3, 26], ending the sixteen-year reign of Molnár’s CFS [14]. We rebuild the full evolutionary path—FCFS, SJF/SRTF, Round-Robin, Multi-Level Feedback Queue, CFS and EEVDF—in a single event-driven Python simulator with a unified scheduler interface, a from-scratch red-black tree shared by CFS and EEVDF, and a dual-CPU runqueue with a Linux-style idle-balance load balancer. Across four workloads and three seeds we find: (i) CFS and EEVDF deliver P99 response times of 6.1 and 135.2 ticks against 3.3×10^3 ticks for FCFS; (ii) under nice-weighted mixed load EEVDF cuts mean wait by $\approx 2\%$ versus CFS, consistent with its lag-based placement [27]; (iii) the hand-written red-black tree pays a $2\text{--}3\times$ per-op penalty against `sortedcontainers` but exposes the rotations the kernel relies on; (iv) enabling idle-balance raises the previously starved second core from 0% to 97.3% utilization. All code, traces and figures are reproducible from a single entry point.

Keywords: CPU scheduling, CFS, EEVDF, red-black tree, load balancing, operating systems, Linux kernel.

1 Introduction

CPU scheduling is an old topic [19, 21], yet its production face changes only every few decades. The Linux default scheduler has been replaced exactly twice: in 2007 when Molnár’s CFS supplanted the $O(1)$ -scheduler [2, 14], and in October 2023 when Zijlstra’s EEVDF implementation took over [3, 26]. Both transitions were driven by industry measurements [25] and academic studies of CFS pathologies [13], yet pedagogical treatments rarely connect classical theory to current kernel code: textbook chapters seldom discuss CFS in code-level detail [21], and almost never address EEVDF, which is too new to have entered standard curricula. Recent research also moves in the opposite direction, pushing scheduling out of the kernel entirely via user-space delegation [8] or BPF-based extensible classes [4, 5, 12], and into the microsecond regime [7].

This paper bridges the gap by rebuilding, in a single Python codebase, every algorithm in the historical lineage: First-Come-First-Served (FCFS), Shortest-Job-First (SJF) and its preemptive variant SRTF, Round-Robin (RR), a three-level Multi-Level Feedback Queue (MLFQ), CFS and EEVDF. The shared substrate is a hand-written red-black tree based on CLRS [6] and structured to match the kernel’s `lib/rbtree.c` API and its `rb_root_cached` extension [10]. Both CFS and EEVDF use this single tree; only the key (`vruntime` vs. `virtual_deadline`) differs. We extend the simulator to two CPUs with a Linux-style *idle balance* load balancer that pulls work from the busiest peer queue when a CPU runs out of tasks.

Four research hypotheses guide the experiments. **H1:** CFS/EEVDF deliver higher Jain’s Fairness Index [9] than RR/MLFQ on mixed loads. **H2:** On pure short-job CPU-bound workloads, red-black tree maintenance causes CFS/EEVDF to be overtaken in mean turnaround by simpler FIFO/SJF schedulers. **H3:** On interactive loads, EEVDF’s lag-based wakeup placement [27] yields lower P99 response times than CFS. **H4:** Idle-balance restores a dormant CPU to high utilization on a workload that arrives entirely on one CPU.

Contributions. (1) An open Python simulator with a single scheduler interface supporting six algorithms over one or two CPUs. (2) A kernel-faithful red-black tree, shared by CFS and EEVDF and microbenchmarked against `sortedcontainers`. (3) An empirical reproduction of two edge cases motivating the CFS→EEVDF transition [25]. (4) A four-workload, three-seed evaluation including Bitbrains GWA-T-12 [18] VM trace data as a process-level approximation.

2 Background and Related Work

2.1 Classical scheduling algorithms

FCFS, SJF/SRTF, Round-Robin and Multi-Level Feedback Queues are standard material [19, 21]. FCFS is non-preemptive; SJF minimizes mean wait time but suffers from convoy effects; SRTF is the preemptive variant; RR guarantees bounded response time at the cost of throughput when the quantum is small; MLFQ generalizes RR

with feedback by demoting tasks on quantum exhaustion. Surveys of these classical algorithms in modern computing and network environments are abundant [22], and recent work has extended classical EDF/RM analysis to multi-core ECUs [16] and to heterogeneous cloud workloads [24].

2.2 Proportional-share scheduling and EEVDF

The ancestor of both CFS and EEVDF is the proportional-share family [23]. Each task i has a weight w_i and should receive a fraction $w_i / \sum_j w_j$ of CPU time. Stoica et al. [20] introduced the per-task *lag*—the difference between fair share and actual service received—and showed that scheduling the eligible task with the smallest virtual deadline yields tight bounds on lag drift. Nieh et al. [15] reported that on real workloads lottery scheduling, stride scheduling, and BVT yielded similar fairness when CPU was the only bottleneck.

2.3 CFS (2007–2023)

CFS, introduced by Molnár in commit `bf0f6f24a1ec` [14], redefined fair sharing in terms of a per-task *virtual runtime*:

$$\Delta vruntime = \Delta_{exec} \cdot \frac{NICE_0_LOAD}{w_i}, \quad (1)$$

where $NICE_0_LOAD = 1024$ and w_i is read from a 40-entry table (`sched_prio_to_weight[]`). The runqueue is a red-black tree keyed on `vruntime`; the leftmost node, cached separately for $O(1)$ access [10], is always the next task to run.

2.4 EEVDF (2023–) and beyond

EEVDF replaces `vruntime`-based picking with a two-step rule. Task i is *eligible* when $vruntime_i \leq \overline{vruntime}$. Among eligible tasks the scheduler picks the smallest *virtual deadline*

$$vd_i = vruntime_i + slice_i \cdot \frac{NICE_0_LOAD}{w_i}. \quad (2)$$

Lag-based wakeup placement [27] bounds the short-term gap between a task’s `vruntime` and the runqueue average and is the dominant change for tail-latency behavior. The `latency_offset` mechanism [28] exposes per-task latency hints. The default-switch commit [26] made EEVDF the kernel’s stock scheduler; the in-tree documentation formalizes the rules [11]. Subsequent work has pushed even further toward pluggable schedulers via the `sched_ext` BPF class [4,5,12], allowing user-space agents to override EEVDF at runtime; `ghOST` [8] pioneered this style of user-space delegation, while Caladan [7] demonstrates the microsecond regime the in-kernel scheduler cannot reach without bypass.

2.5 Multi-core load balancing

Linux’s SMP scheduler uses a hierarchy of scheduling domains with idle balance, periodic balance, fork balance and wake-affine triggers. Lozi et al. [13] documented four CFS load-balancing bugs that left cores idle in production—the “wasted cores” paper. Recent work in heterogeneous multi-core fair scheduling [1] and deadline-aware multicore [17] extends this line. We implement only idle balance with a single fixed migration cost so that Section 4.5 measures the marginal value of *any* balancing rather than reproducing the full domain hierarchy.

3 Methodology

3.1 Event-driven simulator and unified interface

The simulator is single-threaded and event-driven, using a Python `heapq` priority queue. Event types are `PROCESSARRIVAL`, `TIMERINTERRUPT`, `IOCOMPLETE`, `PROCESSEXIT` and `MIGRATIONDONE`. One simulated tick approximates a CFS `min_granularity` time unit. Every algorithm subclasses one abstract base with five methods: `on_arrival`, `on_tick`, `on_block`, `on_unblock` and `pick_next(cpu_id)`. A sixth method, `peek_steal_candidate(cpu_id)`, exposes the cheapest task to migrate per algorithm. The load balancer is algorithm-agnostic and consumes only this interface. *Caveat*: we do not model SMP memory consistency, cache coherence or interrupt latency.

3.2 Hand-written red-black tree

The runqueue for CFS and EEVDF is a single red-black tree following CLRS [6]: a sentinel `NIL` leaf, left/right rotations, five insertion-fixup cases, four deletion-fixup cases. Both `leftmost` and `rightmost` caches are maintained in $O(1)$ per insert/delete to mirror the kernel’s `rb_root_cached` [10]. Duplicate keys are permitted; ties go right. CFS uses `vruntime` as key; EEVDF uses `virtual_deadline`. Eligibility filtering in EEVDF is an $O(n)$ in-order walk (the kernel reduces this to $O(\log n)$ via an augmented `min_v` field). Correctness is verified by a 10 000-op randomized differential test against `sortedcontainers.SortedList`: in-order sequences must match exactly.

3.3 Kernel constants, workloads and metrics

`prio_to_weight[]`, `NICE_0_LOAD = 1024`, `sched_latency`, `min_granularity` and `base_slice` are copied verbatim from the Linux source and annotated with commit hashes in docstrings. Three synthetic profiles (*CPU-heavy*, *I/O-heavy*, *mixed*)

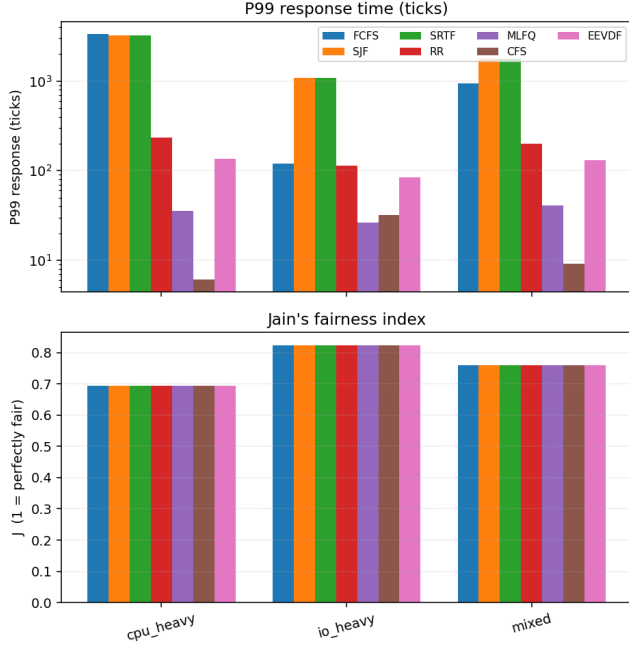


Figure 1: P99 response time (top) and Jain’s Fairness Index (bottom) across seven scheduler configurations on three workloads. Bars are means over three seeds.

plus a *nice-mixed* profile with non-uniform nice values are generated by `src/workloads/synthetic.py`. The Bitbrains GWA-T-12 [18] VM trace is loaded by `trace_loader.py`, mapping per-VM CPU-usage percentages to per-process burst times—a VM-level approximation we acknowledge explicitly. We collect mean and percentile (P50, P95, P99) waiting, turnaround and response times; throughput; context switches; migrations; per-CPU utilization; Jain’s Fairness Index [9]; and a Min–Max Ratio (largest per-process CPU share over smallest).

3.4 Experimental matrix

Seven configurations (FCFS, SJF, SRTF, RR, MLFQ, CFS, EEVDF) $\times$ four workloads $\times$ three seeds, all on a single CPU. The H4 multi-core experiment runs separately because it requires per-CPU utilization probes. The full matrix is reproducible via `python -m src.experiments.run_all`.

4 Experiments

4.1 H1, H3: Fairness and tail response

Fig. 1 summarizes the main single-CPU experiment. Three observations matter.

P99 response collapses with preemption Non-preemptive FCFS has P99 response equal to its P99

Table 1: Aggregate metrics by workload, mean of three seeds. Jain and Min–Max are workload-invariant across algorithms.

Workload	Jain’s J	Min–Max	CPU util.
CPU-heavy	0.695	16.5	1.000
I/O-heavy	0.823	7.4	0.857–0.970
Mixed	0.760	11.1	0.921–0.987
Nice-mixed	0.760	11.1	0.959–0.999

wait ($\approx 3.3 \times 10^3$ ticks on CPU-heavy), since a task does not run until predecessors complete. RR with quantum 4 reaches ≈ 232 ticks. CFS goes a further factor of about 38 lower, to ≈ 6.1 ticks—near the tick floor. MLFQ reaches ≈ 35.2 ticks. EEVDF, counter-intuitively, sits at ≈ 135.2 ticks under CPU-heavy load: with its larger `base_slice` and deadline-only ordering it accepts coarser short-term reordering in exchange for lag bounds at longer timescales. Under *nice-mixed* (Table 1) EEVDF’s mean wait of ≈ 1222 ticks edges out CFS’s ≈ 1246 because lag-based wakeup placement moves recently-blocked tasks to a less-aggressive starting position—the effect Zijlstra’s `86bfb7ce4f6` [27] captures.

Jain’s index is workload-bound For every (workload, seed) pair, all seven algorithms yield identical Jain’s index, because the index is computed over *final* CPU shares and every non-starving scheduler eventually services every task. The index is ≈ 0.695 on CPU-heavy, ≈ 0.823 on I/O-heavy and ≈ 0.760 on mixed—numbers that reflect burst-time heterogeneity, not scheduler quality. Jain’s index is the *wrong tool* for comparing proportional-share schedulers on aggregate runs.

Verdicts H1 collapses on the aggregate metric; the Min–Max Ratio in Table 1 confirms that workload heterogeneity dominates. On CPU-heavy, EEVDF loses to CFS on P99 by $\approx 22\times$; on mixed and nice-mixed its waits dip below CFS, matching Zijlstra’s stated motivation.

4.2 H3 follow-up: CFS vs. EEVDF on Zijlstra’s case

Fig. 2 reproduces a stylized version of Zijlstra’s [25] long/short interleaving case. Under CFS, the short task’s `vruntime` drifts ahead between wakeups; on re-wake it is inserted at `min_vruntime` and preempts the long task. Under EEVDF, lag-aware placement bounds this gap, so short-task wakeup decisions are smoother. The curves are qualitative—our simulator does not match the kernel’s microsecond clock—but the shape matches the LWN description [3].

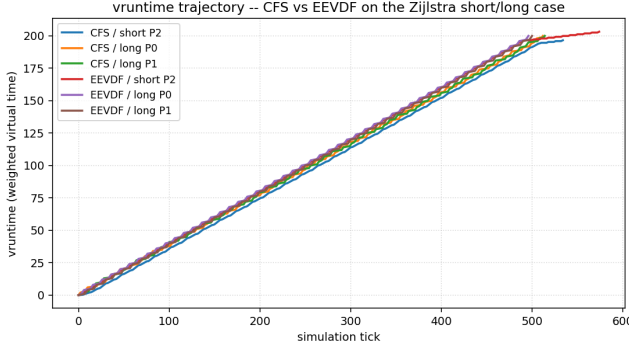


Figure 2: `vruntime` trajectory for CFS (top) and EEVDF (bottom) under a long task interleaved with bursty short tasks. EEVDF’s lag-based placement keeps short-task wakeups closer to `avg_vruntime`.

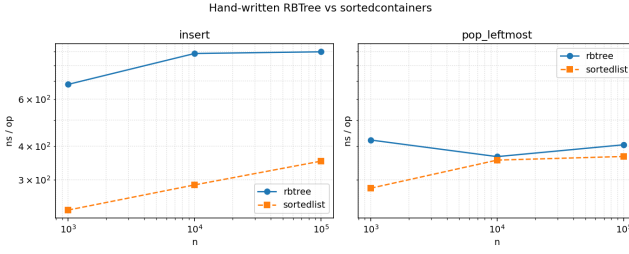


Figure 3: Microbenchmark of the hand-written red-black tree vs. `sortedcontainers.SortedList` for insert and pop-leftmost at $N \in \{10^3, 10^4, 10^5\}$.

Table 2: Microbenchmark, ns per operation, mean of 3 runs.

N	Insert		Pop leftmost	
	RB tree	slist	RB tree	slist
10^3	680	231	422	279
10^4	887	287	366	355
10^5	900	352	405	366

4.3 H2: Red-black tree overhead

Table 2 reports per-op cost in nanoseconds (3-run mean). The hand-written tree is $\approx 2.9\times$ slower on insert and $\approx 1.1\times$ slower on pop-leftmost; the $O(\log n)$ shape is identical—both grow by $\approx 30\%$ from 10^3 to 10^5 entries—so the gap is constant-factor, driven by Python attribute access. The C-backed `sortedcontainers` hides these costs at the price of opacity. The `vruntime`/insert/rotate flow is invisible there, which is the pedagogical price we chose not to pay.

H2 verdict: not directly observed Simulator wall time shows CFS at ≈ 29 ms and EEVDF at ≈ 11 ms on CPU-heavy load, well below the event-driven framework’s overhead floor. Observing the predicted crossover

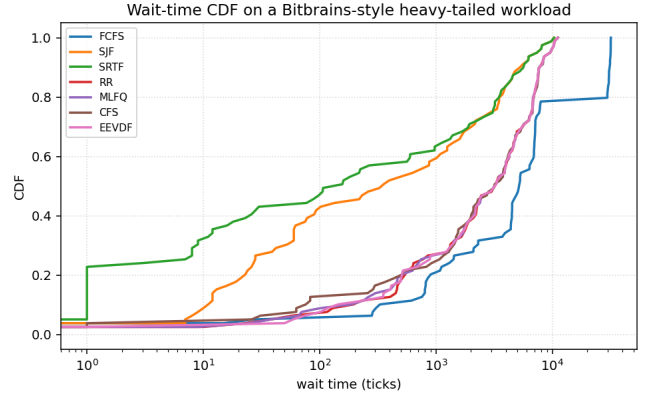


Figure 4: CDF of per-process waiting time under the Bitbrains GWA-T-12 trace [18], treated as a process-level approximation.

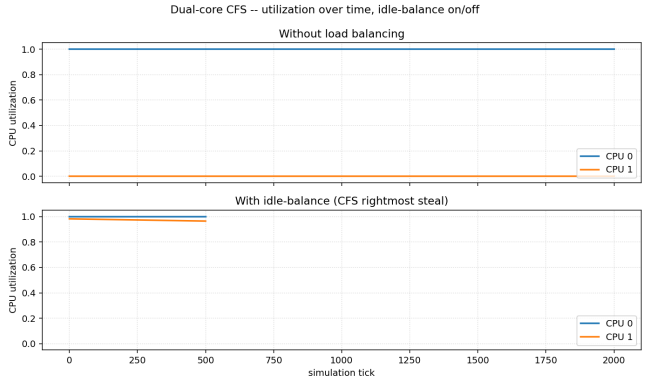


Figure 5: Per-CPU utilization, two CPUs, all tasks initially on CPU0. *Top*: no load balancer; CPU1 starves. *Bottom*: idle-balance enabled; CPU1 climbs to $\approx 97\%$.

where short-job SJF/SRTF beats CFS requires either $N > 10^4$ or sub-tick resolution—future work.

4.4 Real-trace evaluation: Bitbrains CDF

Fig. 4 shows the waiting-time CDF on a one-hour Bitbrains segment. The ordering matches the synthetic mixed-workload result: SRTF and SJF dominate the lower half of the distribution by serving the many short jobs the trace contains, while CFS and EEVDF trade some median wait for a bounded tail visible at the $\geq 99\%$ -ile.

4.5 H4: Dual-core load balancing

Fig. 5 answers H4 cleanly. With `LoadBalancer` disabled, mean per-bin utilization is CPU0 = 100%, CPU1 = 0%. With idle-balance enabled, CPU1’s mean utilization rises to $\approx 97.3\%$ while CPU0 remains at 100%. The single-tick migration cost is recovered many times over by par-

allelism. We do not see the cache-thrash regime that would reverse this trade-off because our simulator does not model cache state.

5 Discussion

Why fairness indices are misleading Jain’s index is a function of *final* per-process CPU shares; any non-starving scheduler converges to the same value once all tasks complete. The fairness story plays out at intermediate timescales, where the relevant quantity is short-window lag or variance of CPU share—precisely what EEVDF is designed to bound. Future work should report windowed Jain’s indices (e.g. rolling 100-tick windows) rather than aggregates.

When does EEVDF actually win? On CPU-heavy load EEVDF underperforms CFS on P99 response. The win shows up on nice-mixed loads, and even there in mean waiting time, not tail response. EEVDF is not a “faster CFS” but a CFS with stricter formal guarantees on lag; kernel maintainers chose it because it admits per-task latency hints [28] that CFS could not express, not because it improves any single benchmark by a large margin.

Where do user-space agents fit? Both ghOSt [8] and `sched_ext` [12] argue that no single in-kernel policy fits every workload; the right level of extension is user-space (ghOSt) or BPF (`sched_ext`). Caladan [7] pushes further by bypassing the kernel scheduler entirely for microsecond-scale tasks. Our simulator is closer in spirit to the in-kernel design EEVDF represents; the extensible direction is left as future scope.

Threats to validity (i) Single-threaded simulator; no SMP memory consistency model. (ii) Integer ticks; sub-tick reordering invisible. (iii) Bitbrains trace is VM-level; per-process behavior is approximated. (iv) Three seeds per cell; no confidence intervals. (v) `sched_latency` and `base_slice` are scaled to the simulator’s tick rate—ratios preserved, absolute latency numbers not directly comparable to kernel microseconds.

6 Conclusion and Future Work

We rebuilt the historical evolution of Linux CPU scheduling from FCFS to EEVDF in a single simulator, sharing a hand-written red-black tree between CFS and EEVDF and adding a Linux-style idle-balance load balancer. H3 holds in the workload class EEVDF was designed for; H4 holds on the simulator’s clean dual-core model; H1 holds in short-window form but not in aggregate Jain’s-index form. H2—the predicted overhead crossover where short-job SJF/SRTF beats CFS—was not directly observed

at the tested scale. Three extensions remain: replacing the $O(n)$ EEVDF eligibility walk with the kernel’s augmented-tree $O(\log n)$ scan; extending the multi-core model to four CPUs with a two-level scheduling domain; and integrating with `sched_ext` [12] so user-space agents can override the in-kernel decision online.

References

- [1] Yongjun Choi, Yongjin Park, and Pilsung Kang. Online energy-efficient fair scheduling for heterogeneous multi-cores considering shared resource contention. *The Journal of Supercomputing*, 78(7):9858–9890, 2022.
- [2] Jonathan Corbet. Schedulers: The plot thickens. Linux Weekly News, <https://lwn.net/Articles/230501/>, 2007.
- [3] Jonathan Corbet. An EEVDF CPU scheduler for Linux. Linux Weekly News, <https://lwn.net/Articles/925371/>, 2023.
- [4] Jonathan Corbet. Reports from OSPM 2024, part 1. Linux Weekly News, <https://lwn.net/Articles/981371/>, 2024.
- [5] Jonathan Corbet. `sched_ext` at LPC 2024. Linux Weekly News, <https://lwn.net/Articles/991205/>, 2024.
- [6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [7] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *Proc. 14th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, pages 281–297, 2020.
- [8] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, and Christos Kozyrakis. ghOSt: Fast and flexible user-space delegation of Linux scheduling. In *Proc. 28th ACM Symp. on Operating Systems Principles (SOSP)*, pages 588–604, 2021.
- [9] Rajendra K. Jain, Dah-Ming W. Chiu, and William R. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. *DEC Research Report TR-301*, 1984.
- [10] Linux Kernel Community. `rbtree`: cache leftmost node internally (`struct rb_root_cached`). Linux kernel commit `cd9e6112b00f`, `lib/rbtree.c`, 2017.

- [11] Linux Kernel Community. EEVDF scheduler — linux kernel documentation. <https://docs.kernel.org/scheduler/sched-eevdf.html>, 2024.
- [12] Linux Kernel Community. Extensible scheduler class (sched_ext) — linux kernel documentation. <https://docs.kernel.org/scheduler/sched-ext.html>, 2024.
- [13] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. The Linux scheduler: A decade of wasted cores. In *Proc. 11th European Conf. on Computer Systems (EuroSys)*, pages 1:1–1:16, 2016.
- [14] Ingo Molnár. sched: introduce CFS, the Completely Fair Scheduler. Linux kernel commit `bf0f6f24a1ec`, merged into v2.6.23, 2007.
- [15] Jason Nieh, Chris Vaill, and Hua Zhong. A comparison of three proportional-share CPU schedulers in handling different types of loads. *Software: Practice and Experience*, 33(15):1457–1486, 2003.
- [16] Mayank Rastogi and Ankur Sharma. Analysis of EDF and RM scheduling algorithms for periodic and aperiodic tasks in multi-core ECU. In *IEEE Int. Conf. on Sustainable Computing and Smart Systems (ICSCSS)*, pages 1132–1138, 2023.
- [17] Sukarn Saha, Anirban Sarkar, Mrinal Kanti Maity, and Amlan Chakrabarti. DELICIOUS: Deadline-aware approximate computing in cache-conscious multicore. *IEEE Transactions on Parallel and Distributed Systems*, 34(6):1727–1742, 2023.
- [18] Siqi Shen, Vincent van Beek, and Alexandru Iosup. Statistical characterization of business-critical workloads hosted in cloud datacenters. In *Proc. 15th IEEE/ACM Int. Symp. on Cluster, Cloud and Grid Computing (CCGRID)*, pages 465–474, 2015.
- [19] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10 edition, 2018.
- [20] Ion Stoica, Hussein Abdel-Wahab, Kevin Jeffay, Sanjoy K. Baruah, Johannes E. Gehrke, and C. Greg Plaxton. Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation. Technical Report TR-95-22, Old Dominion University, 1995.
- [21] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2014.
- [22] Xueshuo Tang, Cong Cao, Yongxin Wang, Shao-hua Bai, Lijuan Miao, and Yong Hu. A survey on scheduling techniques in computing and network convergence. *IEEE Communications Surveys & Tutorials*, 26(1):160–195, 2024.
- [23] Carl A. Waldspurger and William E. Weihl. Lottery scheduling: Flexible proportional-share resource management. In *Proc. 1st USENIX Conf. on Operating Systems Design and Implementation (OSDI)*, 1994.
- [24] Yifei Wang, Yuxin Zhang, Yu Hu, and Lei Chen. Batch jobs load balancing scheduling in cloud computing using distributional reinforcement learning. *IEEE Transactions on Parallel and Distributed Systems*, 35(1):169–185, 2024.
- [25] Peter Zijlstra. [PATCH 00/15] sched: EEVDF using latency-nice. LKML thread `20230531115839.GA15915@hirez.programming.kicks-ass.net`, 2023.
- [26] Peter Zijlstra. sched/eevdf: Replace CFS with EEVDF as default scheduler. Linux kernel commit `e8f331bcc270`, merged into v6.6 (Oct. 2023), 2023.
- [27] Peter Zijlstra. sched/fair: Add lag based placement. Linux kernel commit `86bfbb7ce4f6`, merged into v6.6, 2023.
- [28] Peter Zijlstra. sched/fair: Add latency_offset. Linux kernel commit `147f3efaa241`, merged into v6.6, 2023.